```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
```

```
2026-04-01 10:28:05.359005: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuF
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1775039285.737716      55 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register
E0000 00:00:1775039285.849277      55 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to registe
W0000 00:00:1775039286.796798      55 computation_placer.cc:177] computation placer already registered. Please ch
W0000 00:00:1775039286.796840      55 computation_placer.cc:177] computation placer already registered. Please ch
W0000 00:00:1775039286.796843      55 computation_placer.cc:177] computation placer already registered. Please ch
W0000 00:00:1775039286.796845      55 computation_placer.cc:177] computation placer already registered. Please ch
```

```
data = """artificial intelligence systems learn patterns from data.
sequence models process information step by step.
recurrent neural networks are useful for sequence prediction.
lstm networks handle long term dependencies.

deep learning models improve sequence learning.
generative models create new samples from learned patterns.
language models predict the next word in a sentence.
sequence generation is used in chatbots and assistants.

machine learning helps computers learn automatically.
training data improves model accuracy.
neural networks simulate human brain structures.
optimization algorithms improve learning efficiency.

technology is transforming modern education.
online learning platforms use artificial intelligence.
students benefit from intelligent tutoring systems.
automation improves productivity and decision making."""
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])

total_words = len(tokenizer.word_index) + 1
print(total_words)
```

```
78
```

```
input_sequences = []

for line in data.split("\n"):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        n_gram_seq = token_list[:i+1]
        input_sequences.append(n_gram_seq)

max_seq_len = max(len(x) for x in input_sequences)

input_sequences = pad_sequences(input_sequences, maxlen=max_seq_len, padding='pre')

input_sequences = np.array(input_sequences)
```

```
X = input_sequences[:, :-1]
y = input_sequences[:, -1]

y = tf.keras.utils.to_categorical(y, num_classes=total_words)
```

```
model = Sequential([
    Embedding(total_words, 100),
    LSTM(150),
    Dense(total_words, activation='softmax')
])

model.build(input_shape=(None, max_seq_len-1))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 8, 100)	7,800
lstm_1 (LSTM)	(None, 150)	150,600
dense_1 (Dense)	(None, 78)	11,778

Total params: 170,178 (664.76 KB)
Trainable params: 170,178 (664.76 KB)
Non-trainable params: 0 (0.00 B)

```
model.fit(X, y, epochs=100, verbose=1)
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9442 - loss: 0.5727
Epoch 73/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9364 - loss: 0.5763
Epoch 74/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9364 - loss: 0.5447
Epoch 75/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9421 - loss: 0.5465
Epoch 76/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9656 - loss: 0.4902
Epoch 77/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9460 - loss: 0.5135
Epoch 78/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9478 - loss: 0.5261
Epoch 79/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9634 - loss: 0.4913
Epoch 80/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9734 - loss: 0.4440
Epoch 81/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9556 - loss: 0.4614
Epoch 82/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9769 - loss: 0.4353
Epoch 83/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9634 - loss: 0.4173
Epoch 84/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9808 - loss: 0.3962
Epoch 85/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9847 - loss: 0.4061
Epoch 86/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9730 - loss: 0.3789
Epoch 87/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9730 - loss: 0.3789
Epoch 88/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9826 - loss: 0.3612
Epoch 89/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9826 - loss: 0.3627
Epoch 90/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9904 - loss: 0.3641
Epoch 91/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9886 - loss: 0.3326
Epoch 92/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9943 - loss: 0.3204
Epoch 93/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9826 - loss: 0.3281
Epoch 94/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9904 - loss: 0.3223
Epoch 95/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9943 - loss: 0.2989
Epoch 96/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9826 - loss: 0.3334
Epoch 97/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9847 - loss: 0.2867
Epoch 98/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9943 - loss: 0.2906
Epoch 99/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9904 - loss: 0.2830
Epoch 100/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9904 - loss: 0.2761
<keras.src.callbacks.history.History at 0x7a81a0d500e0>
```

```
def generate_text(seed_text, next_words=10):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_seq_len-1, padding='pre')

        predicted = np.argmax(model.predict(token_list), axis=-1)[0]

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break

    return seed_text
```

```
print(generate_text("machine learning", 10))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 201ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step
machine learning helps computers learn automatically automatically automatically automatically patterns data data
```

Component II: Transformer Model

```
from tensorflow.keras.layers import Layer, Dense, LayerNormalization, Dropout
```

```
class PositionalEncoding(Layer):
    def __init__(self, max_len, d_model):
        super().__init__()
        self.pos_encoding = self.positional_encoding(max_len, d_model)

    def positional_encoding(self, position, d_model):
        angles = np.arange(position)[:, np.newaxis] / np.power(10000, (2 * (np.arange(d_model)[np.newaxis, :]//2
        angles[:, 0::2] = np.sin(angles[:, 0::2])
        angles[:, 1::2] = np.cos(angles[:, 1::2])
        return tf.cast(angles[np.newaxis, ...], dtype=tf.float32)

    def call(self, inputs):
        return inputs + self.pos_encoding[:, :tf.shape(inputs)[1], :]
```

```
class TransformerBlock(Layer):
    def __init__(self, d_model, num_heads, dff):
        super().__init__()
        self.att = tf.keras.layers.MultiHeadAttention(num_heads=num_heads, key_dim=d_model)
        self.ffn = Sequential([
            Dense(dff, activation='relu'),
            Dense(d_model)
        ])
        self.norm1 = LayerNormalization()
        self.norm2 = LayerNormalization()

    def call(self, x):
        attn_output = self.att(x, x)
        out1 = self.norm1(x + attn_output)
        ffn_output = self.ffn(out1)
        return self.norm2(out1 + ffn_output)
```

```
inputs = tf.keras.Input(shape=(max_seq_len-1,))
x = Embedding(total_words, 128)(inputs)
x = PositionalEncoding(max_seq_len, 128)(x)
x = TransformerBlock(128, 4, 128)(x)
x = tf.keras.layers.GlobalAveragePooling1D()(x)
outputs = Dense(total_words, activation='softmax')(x)

transformer_model = tf.keras.Model(inputs, outputs)

transformer_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

transformer_model.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 8)	0
embedding_2 (Embedding)	(None, 8, 128)	9,984
positional_encoding (PositionalEncoding)	(None, 8, 128)	0
transformer_block (TransformerBlock)	(None, 8, 128)	297,344
global_average_pooling1d (GlobalAveragePooling1D)	(None, 128)	0
dense_4 (Dense)	(None, 78)	10,062

Total params: 317,390 (1.21 MB)
Trainable params: 317,390 (1.21 MB)
Non-trainable params: 0 (0.00 B)

```
transformer_model.fit(X, y, epochs=100, verbose=1)
```

```
Epoch 73/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9673 - loss: 0.0780
Epoch 74/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9847 - loss: 0.0692
Epoch 75/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9712 - loss: 0.0725
Epoch 76/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9695 - loss: 0.0642
Epoch 77/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9847 - loss: 0.0603
Epoch 78/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9751 - loss: 0.0762
Epoch 79/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9847 - loss: 0.0614
Epoch 80/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9595 - loss: 0.0870
Epoch 81/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9673 - loss: 0.0816
Epoch 82/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9499 - loss: 0.0842
Epoch 83/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9730 - loss: 0.0887
Epoch 84/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9830 - loss: 0.0655
Epoch 85/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9673 - loss: 0.0759
Epoch 86/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9652 - loss: 0.0734
Epoch 87/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9673 - loss: 0.0704
Epoch 88/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9595 - loss: 0.0826
Epoch 89/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9673 - loss: 0.0703
Epoch 90/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9673 - loss: 0.0543
Epoch 91/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9751 - loss: 0.0568
Epoch 92/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9769 - loss: 0.0649
Epoch 93/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9808 - loss: 0.0640
Epoch 94/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9808 - loss: 0.0471
Epoch 95/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9904 - loss: 0.0550
Epoch 96/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9826 - loss: 0.0730
Epoch 97/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9652 - loss: 0.0729
Epoch 98/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9826 - loss: 0.0677
Epoch 99/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9826 - loss: 0.0621
Epoch 100/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9904 - loss: 0.0554
<keras.src.callbacks.history.History at 0x7a80ec1188c0>
```

```
def generate_text_transformer(seed_text, next_words=10):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_seq_len-1, padding='pre')
```

```
        predicted = np.argmax(transformer_model.predict(token_list), axis=-1)[0]

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break

    return seed_text

print(generate_text_transformer("Deep learning", 10))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
Deep learning models improve sequence learning helps computers learn automatically from data
```

Start coding or generate with AI.